

Cross Challenge — 3 Casos

Banco / Turismo corporativo / Seguros de autos, resueltos con ADD

Ingeniería de Software II · ITBA · 2026-1C · Práctica de cátedra

RESUMEN

Tres casos breves (Banco/minitiendas PYME, Turismo empresarial, Seguros de autos) resueltos con el flujo *Attribute Driven Design*. El insight del "cross" no es la arquitectura correcta de cada uno, sino la divergencia: mismo enunciado, arquitecturas distintas según qué driver priorizó cada equipo (source: cross-challenge-3-casos.md).

§0 Cómo se resuelve cada caso (flujo ADD)

DEFINICIÓN

Cross Challenge: tres casos cortos pensados para trabajar **en paralelo** por equipos distintos y luego **cruzar** (de ahí "cross") las soluciones, comparando decisiones arquitectónicas frente a restricciones similares (source: cross-challenge-3-casos.md).

Cada caso ejercita el mismo flujo de *Attribute Driven Design* ([[Attribute Driven Design]]) sobre un dominio distinto:

1. **Identificar stakeholders.**
2. **Priorizar atributos** de calidad (tabla ISO 25000) → chips.
3. **Construir** [[Árbol de utilidad]]: 3-5 escenarios hoja por atributo dominante.
4. **Elegir** [[Estilos arquitectónicos]] justificando contra los drivers.
5. **Cruzar** resultados entre equipos: ¿qué driver hizo dominar cada uno? (source: cross-challenge-3-casos.md)

EN EL PARCIAL

No saltes al dibujo de cajas antes de articular stakeholders y atributos: el orden ADD (drivers → escenarios → estilo → diagrama) es lo que se puntúa. Una respuesta mínima = atributos priorizados + 2 trade-offs explícitos + 1 diagrama con componentes nombrados + escenarios.

§1 Caso 1 — Banco / minitiendas PYME

Dominio: un banco lanza una línea para *minitiendas PYME* — comercios chicos que necesitan medios de cobro electrónico (POS, link de pago, QR) integrados con su cuenta bancaria (source: cross-challenge-3-casos.md).

Stakeholders

Banco (riesgo, compliance) · dueños de comercio (cobrar ya, sin IT interno) · clientes finales del comercio · core bancario legacy · regulador.

Atributos priorizados

SECURITY **AVAILABILITY** **USABILITY** **SCALABILITY** **TIME-TO-MARKET**

- **SECURITY** — se maneja dinero; máxima prioridad.
- **AVAILABILITY** alta — un POS caído = ventas perdidas del comercio.
- **USABILITY** — PYMEs sin IT interno: onboarding y operación simples.
- **SCALABILITY** — potencialmente miles de tiendas.
- **TIME-TO-MARKET** — compite con fintechs ágiles (source: cross-challenge-3-casos.md).

Árbol / escenarios

- **Security:** un atacante intenta replicar una transacción de cobro → el sistema la rechaza por idempotencia + firma; 0 cobros duplicados maliciosos.
- **Availability:** cae el core bancario en horario pico → el POS sigue cobrando en modo offline-first y concilia al reconectar.
- **Usability:** un dueño sin conocimientos de IT da de alta su comercio → operativo en < 10 min sin soporte.
- **Scalability:** se suman 5.000 comercios en una campaña → latencia de cobro estable, escalado horizontal de los servicios de pago.

Diagrama de componentes

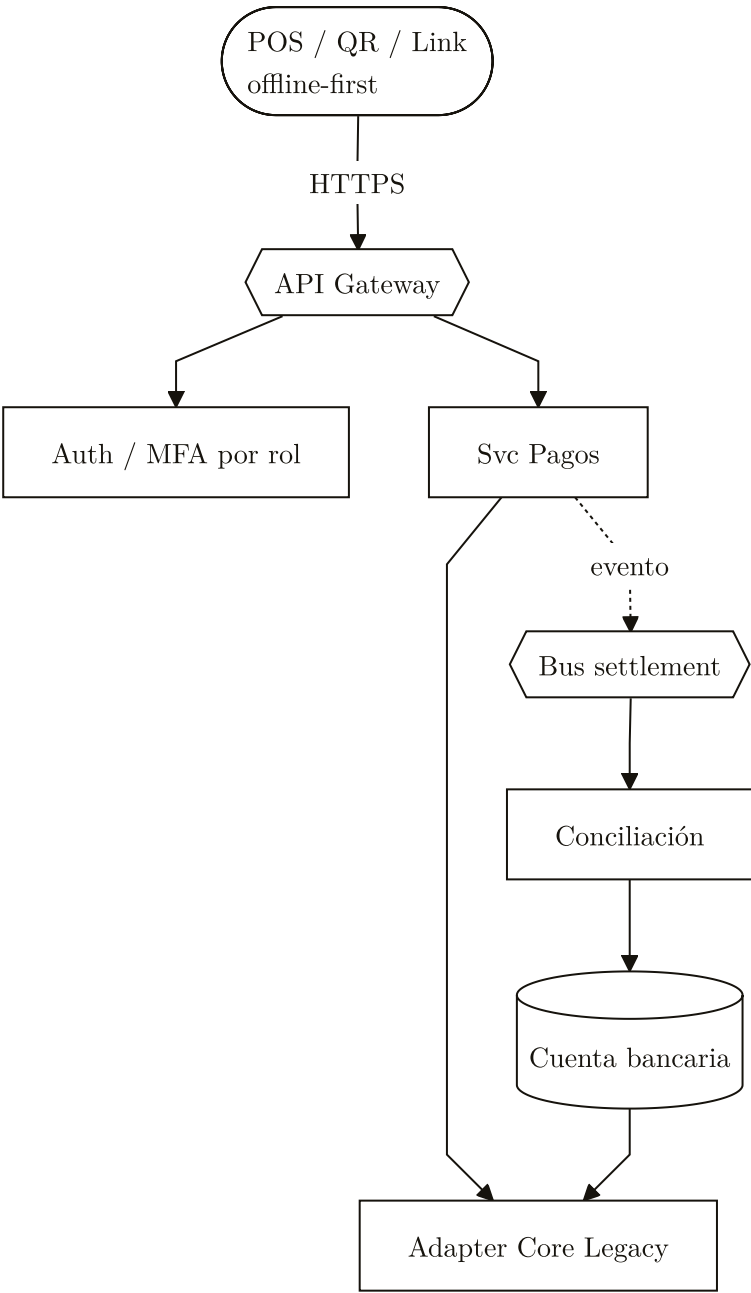


FIGURA 1. Caso Banco PYME: cobro offline-first vía gateway con MFA por rol, pagos desacoplados del settlement por eventos y conciliación contra el core legacy.

TRADE-OFF

Security ↔ Usability: MFA agrega fricción a PYMEs sin IT. Se resuelve aplicándolo selectivamente por rol/monto, no a todo cobro (source: cross-challenge-3-casos.md).

TRADE-OFF

Integración con core legacy ↔ Time-to-market: acoplarse al core bancario es lento. Se aísla con un **adapter** y se publica settlement por eventos para no bloquear el time-to-market del front.

Estilos candidatos

Estilo	Por qué
Microservicios + API Gateway	Aísla pagos/auth/settlement; escala por servicio (Scalability) y permite iterar rápido (Time-to-market).
Event-driven (settlement)	Desacopla cobro de conciliación; absorbe picos sin bloquear al comercio (Availability).
Offline-first (POS físico)	El POS cobra aunque caiga el backend; concilia al reconectar (Availability).

CUÁNDO NO • CUIDADO

Error frecuente: sub-estimar **auditability** en un caso regulado (manejo de dinero) y "usar microservicios porque sí" sin justificar contra los drivers (source: cross-challenge-3-casos.md).

§2 Caso 2 — Turismo empresarial

Dominio: plataforma de *gestión de viajes corporativos* — empleados solicitan viajes, flujos de aprobación, booking con múltiples proveedores, facturación corporativa (source: cross-challenge-3-casos.md).

Stakeholders

Empresa cliente (políticas, presupuesto) · empleados viajeros · aprobadores · proveedores (GDS, hoteleras, aerolíneas) · finanzas/compliance corporativo.

Atributos priorizados

INTEROPERABILITY WORKFLOW CONFIGURABILITY AUDITABILITY PERFORMANCE AVAILABILITY

- **INTEROPERABILITY** — GDS (Amadeus, Sabre), hoteleras, aerolíneas. Es el driver dominante.
- **WORKFLOW CONFIGURABILITY** — cada empresa tiene reglas de aprobación distintas.
- **AUDITABILITY** — expenses corporativos con compliance.
- **PERFORMANCE** — búsquedas de vuelos razonablemente rápidas.
- **AVAILABILITY** — útil 24×7 para viajeros en cualquier zona horaria (source: cross-challenge-3-casos.md).

Árbol / escenarios

- **Interoperability:** se incorpora un nuevo GDS → se conecta vía un adapter del broker sin tocar el core de booking.
- **Workflow configurability:** una empresa exige doble aprobación > cierto monto → se configura sin desplegar código.
- **Auditability:** auditoría pide la traza de aprobación de un gasto → se reconstruye quién aprobó qué y cuándo.
- **Performance:** búsqueda multi-proveedor de vuelos → resultados consolidados en tiempo aceptable pese a fan-out a varios GDS.

Diagrama de componentes

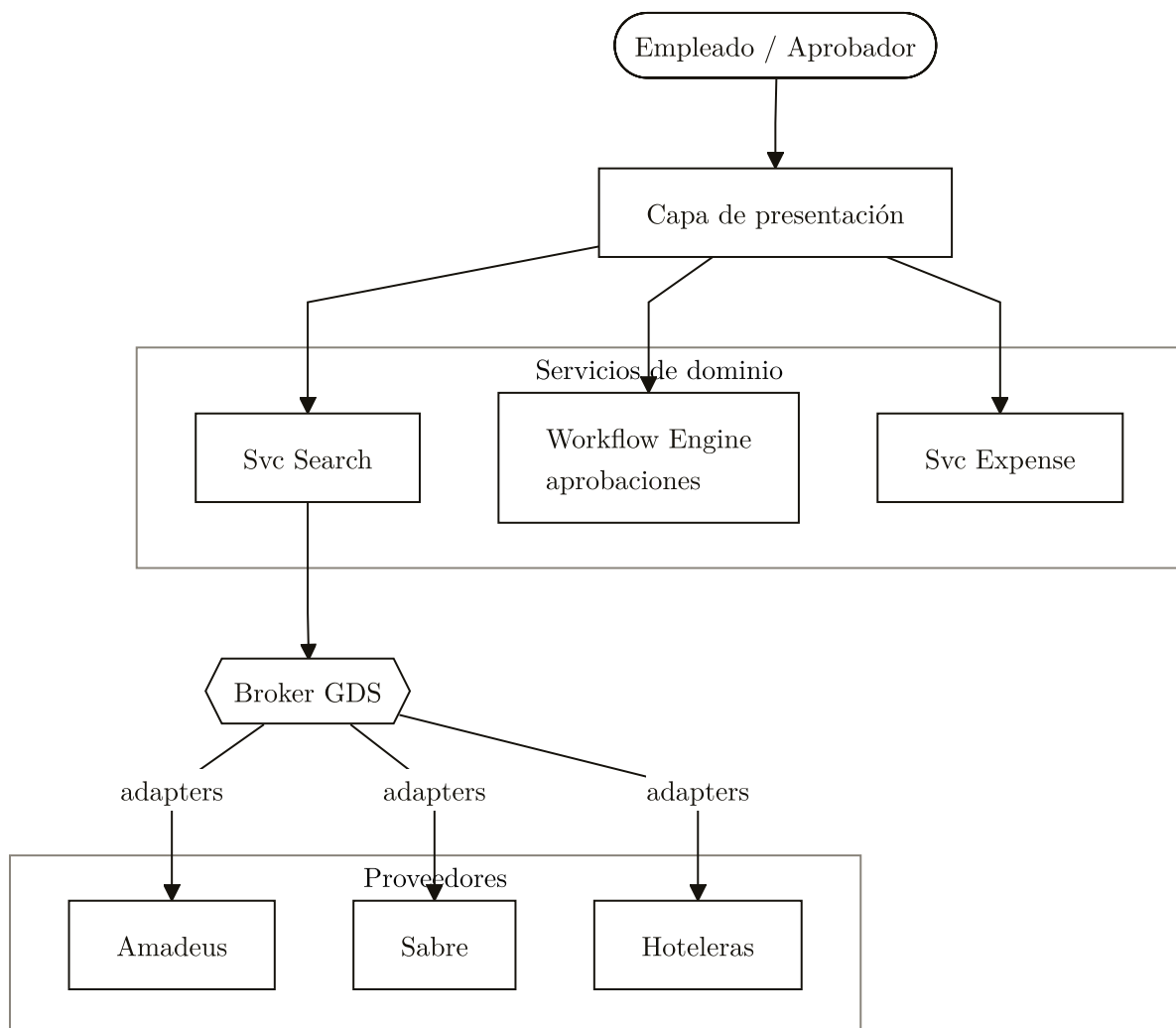


FIGURA 2. Caso Turismo: presentación sobre servicios de search, workflow y expense, con un Broker GDS que normaliza proveedores heterogéneos mediante adapters (driver dominante: interoperability).

TRADE-OFF

Integración profunda ↔ Agilidad de onboarding: integrar a fondo cada proveedor da mejor UX pero frena sumar proveedores nuevos. El patrón **Broker** con adapters por proveedor resuelve la tensión (source: cross-challenge-3-casos.md; Hohpe & Woolf, EIP).

TRADE-OFF

Personalización por empresa ↔ Simplicidad de plataforma: reglas distintas por cliente complican el core. Se externaliza al **workflow engine** configurable en vez de ramificar el código.

Estilos candidatos

Estilo	Por qué
Layers + Broker hacia GDS	El Broker normaliza N proveedores heterogéneos: máximo Interoperability.
Workflow engine	Aprobaciones configurables por empresa sin tocar código (Workflow configurability + Auditability).
Microservicios por dominio	Separa search / booking / expense para escalar y evolucionar independientes.

CUÁNDO NO • CUIDADO

Error frecuente: copiar microservicios de moda y olvidar que aquí el driver dominante es **integration** — sin un broker bien diseñado, la fragmentación en servicios no agrega nada (source: cross-challenge-3-casos.md).

§3 Caso 3 — Seguros de autos

Dominio: *gestión de seguros automotor* — cotización, emisión de pólizas, endosos, gestión de siniestros, peritajes, pagos de indemnizaciones (source: cross-challenge-3-casos.md).

Stakeholders

Asegurado · perito · banco (pagos) · regulador (compliance aseguradora) · talleres y proveedores de inspecciones · área actuarial.

Atributos priorizados

AUDITABILITY

RELIABILITY

INTEGRATION

WORKFLOW

SCALABILITY

- **AUDITABILITY** extrema — regulación aseguradora.
- **RELIABILITY** de los cálculos actuariales.
- **INTEGRATION** con proveedores de inspecciones, talleres, peritos.
- **WORKFLOW** complejo en siniestros — muchos estados, muchos actores.
- **SCALABILITY** con fluctuaciones estacionales (source: cross-challenge-3-casos.md).

Árbol / escenarios

- **Auditability:** el regulador pide la historia completa de una póliza → se reconstruye estado por estado desde el log de eventos.
- **Reliability:** se recalcula una prima actuarial → resultado determinista y trazable, sin discrepancias.
- **Workflow:** un siniestro pasa por denuncia → peritaje → ajuste → pago → el motor de workflow guía cada transición con sus actores.
- **Scalability:** pico estacional de cotizaciones → la lectura/cotización escala sin afectar la emisión.

Diagrama de componentes

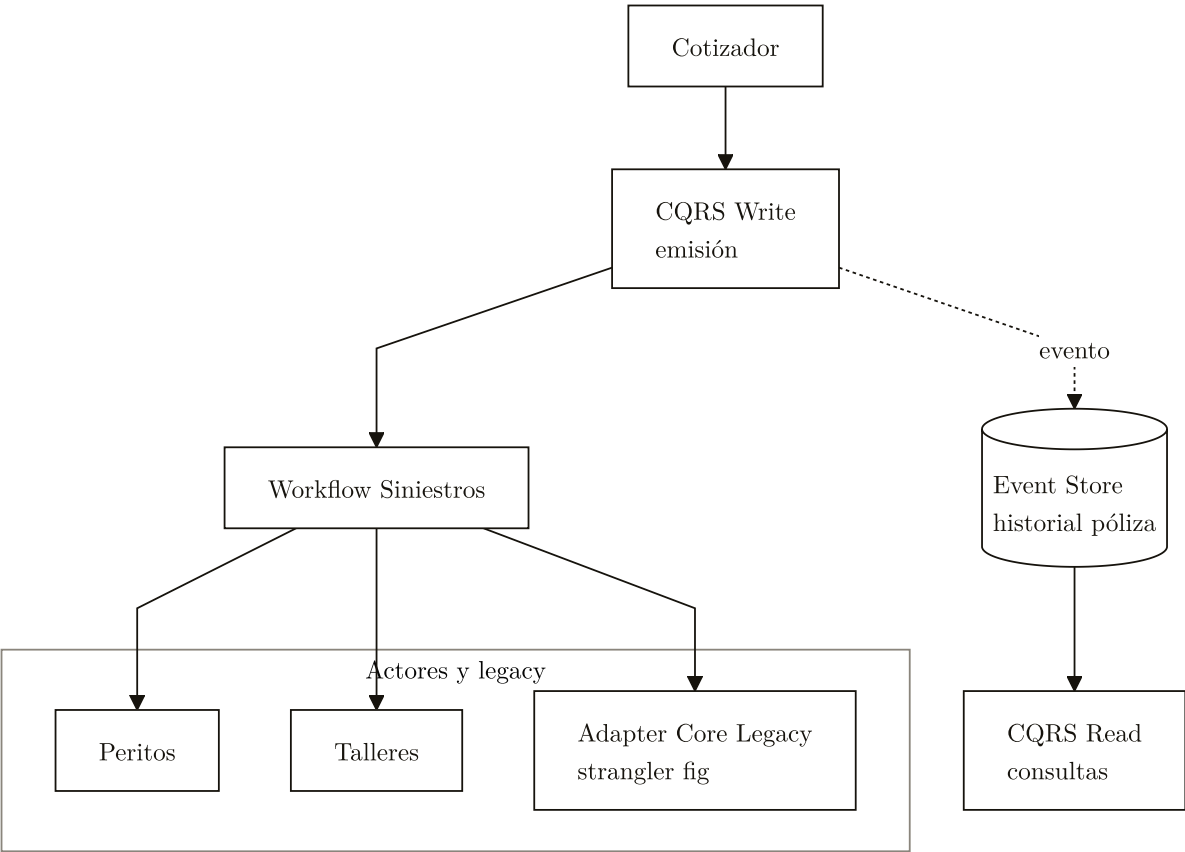


FIGURA 3. Caso Seguros: emisión por CQRS Write que publica al Event Store inmutable, lecturas por CQRS Read, y workflow de siniestros que coordina peritos, talleres y el core legacy estrangulado.

TRADE-OFF

Time-to-decision ↔ Precisión actuarial: cotizar rápido (Performance) tensiona con precisión (Reliability). Se separa con **CQRS**: cotización/consulta optimizada para latencia, emisión optimizada para corrección (source: cross-challenge-3-casos.md).

TRADE-OFF

Workflow rígido ↔ Flexibilidad: un flujo rígido da Auditability pero ahoga los casos atípicos de siniestro. Se balancea con un workflow engine que permite excepciones **auditadas**, no informales.

Estilos candidatos

Estilo	Por qué
Strangler fig sobre core legacy	Moderniza incrementalmente sin reescribir el sistema asegurador existente.
Event sourcing	Historial inmutable de la póliza: máxima Auditability ante el regulador.
CQRS	Separa emisión (escritura, precisa) de consulta/cotización (lectura, escalable).
Workflow engine	Modela los múltiples estados y actores del siniestro.

CUÁNDO NO • CUIDADO

Error frecuente: ignorar las restricciones de integración con el **legacy** y sub-estimar auditability — en seguros, perder la traza regulatoria invalida toda la solución (source: cross-challenge-3-casos.md).

§4 El valor del "cross" y el Architecture Business Cycle

El insight principal **no** es *la* arquitectura "correcta" de cada caso — es **observar la divergencia**: mismo enunciado, arquitecturas distintas según qué driver priorizó cada equipo (source: cross-challenge-3-casos.md).

TABLA 1. Cómo el driver dominante define la arquitectura

Caso	Driver que domina	Decisión arquitectónica resultante
Banco PYME	Security + Availability	Microservicios + event-driven + offline-first
Turismo	Interoperability	Broker hacia GDS + workflow engine
Seguros	Auditability + Reliability	Event sourcing + CQRS + strangler fig

FUENTE

Si dos equipos resuelven el mismo caso y eligen distinto driver dominante, llegan a arquitecturas distintas — y ambas pueden ser correctas. Es la lección central del cross ([Architecture Business Cycle]).

CUÁNDO SÍ • VENTAJAS

Qué demuestra el cruce: el *Architecture Business Cycle* ([Architecture Business Cycle]) — equipos con distinta "experience" y "technical environment" llegan a decisiones distintas frente al mismo problema. La arquitectura no sale sólo del problema, sino del contexto del equipo y la organización (Bass, Clements, Kazman, SAIP).

FUENTE

No hay arquitectura universal: cada solución es óptima sólo respecto del driver que el equipo eligió priorizar (Brooks, No Silver Bullet; ver [[Arquitectura de software — definición]]).

Patrones de error a evitar (transversal)

CUÁNDO NO • CUIDADO

- **Copiar arquitecturas de moda** sin justificar contra los drivers ("usamos microservicios porque sí").
- **Ignorar restricciones** de integración con legacy.
- **Sub-estimar auditability** en los casos regulados (banco, seguros).
- **Olvidar scalability** o definirla sólo como "horizontal" sin aterrizar el escenario (source: cross-challenge-3-casos.md).

REGLA DE ORO

Ante un caso del cross, preguntá siempre primero: ¿cuál es el driver dominante? La arquitectura es la consecuencia de esa elección, no al revés.

Fuentes: Bass, Clements, Kazman — Software Architecture in Practice; Richards, Ford — Fundamentals of Software Architecture; Hohpe, Woolf — Enterprise Integration Patterns (source: cross-challenge-3-casos.md).